

Beyond Predefined Interactions: Enhancing Gesture-Based Flexible Robot Interaction through Air-Writing and Large Models Inference

Huizi Han

Department of Computing
The Hong Kong Polytechnic University
Hong Kong, China
hanhuizi01@foxmail.com

Wei Ren

Department of Computer Science
City University of Hong Kong
Hong Kong, China
weiren_thu@163.com

Wenting He

Department of Computing
The Hong Kong Polytechnic University
Hong Kong, China
wenting.he@polyu.edu.hk

Jiannong Cao

The Hong Kong Polytechnic University
Hong Kong, China
jiannong.cao@polyu.edu.hk

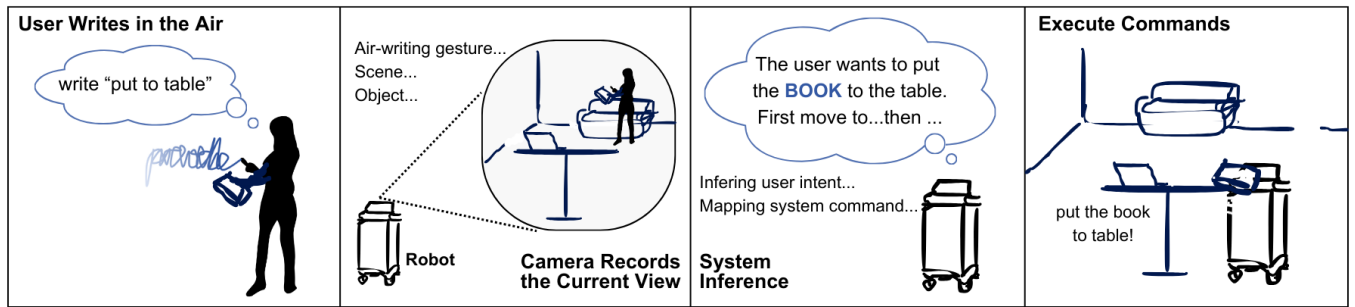


Figure 1: The user interacts with the robot using GesFlexiBot.

Abstract

Gesture is widely used in human-robot interaction, particularly in scenarios where speech is impractical, such as quiet rooms where verbal commands are restricted, noisy factory floors, or for users with speech impairments. However, most existing systems rely on predefined gestures, which limit the flexibility of user expression and increase memory load. In this paper, we present GesFlexiBot, a flexible gesture-based system that enables natural robot control through air-writing. Users can write letters or words in free space to express their wishes, which GesFlexiBot accurately interprets their intents and maps to commands, providing greater freedom than traditional gestures. The system combines camera-based gesture recognition for decoding text with context analysis, where VLMs and object detection provide environmental context. An LLM then infers the correct gesture inputs and user intent, mapping them to executable commands. GesFlexiBot enhances continuous letter decoding and resolves ambiguous relationships between intents and commands, enabling flexible and intuitive robot control.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '25, Yokohama, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1395-8/25/04

<https://doi.org/10.1145/3706599.3719864>

Preliminary experiments demonstrated its capability in three classic human-robot interaction tasks, highlighting its potential for real-world applications.

CCS Concepts

• **Human-centered computing** → **Human computer interaction (HCI); Gestural input.**

Keywords

Gesture-based interaction, Air-writing gestures, Human-robot interaction, Intent inference, Command mapping

ACM Reference Format:

Huizi Han, Wenting He, Wei Ren, and Jiannong Cao. 2025. Beyond Predefined Interactions: Enhancing Gesture-Based Flexible Robot Interaction through Air-Writing and Large Models Inference. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*, April 26–May 01, 2025, Yokohama, Japan. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3706599.3719864>

1 INTRODUCTION

Interacting with robots through gestures offers a natural and intuitive interface, especially in environments where speech is impractical, such as quiet spaces or noisy settings. Gesture-based control allows users to express their intent without relying on keyboards

or touchscreens. However, current systems often rely on predefined gestures mapped to specific commands, limiting flexibility and requiring users to remember multiple gestures. This increases cognitive load, especially when switching tasks or adapting to different situations. Furthermore, the ability of systems to interpret gestures and translate them into meaningful system actions is often constrained by the static nature of predefined gestures, which do not account for the variability and fluidity of human expression.

To overcome these limitations, we propose shifting the complexity to the system itself. Instead of burdening users with the need to memorize numerous gestures, the system can infer user intent from more flexible inputs, such as free-form gestures, and dynamically adapt to changing contexts. This approach leverages the recent advances in Visual Language Models (VLMs) and Large Language Models (LLMs), which can process complex and continuous inputs while adapting to various environments and tasks. By doing so, the system can offer a more intuitive interface that enhances its adaptability to different tasks and environments, while simultaneously reducing cognitive load and providing users with greater flexibility in expressing their needs.

In this paper, we present GesFlexiBot, a flexible gesture-based robot interaction system that enables control through air-writing gestures. Users can express diverse intents by writing letters or words in free space, offering more freedom than traditional gestures. By using visual language models (VLMs) and large language models (LLMs), GesFlexiBot overcomes the challenge of low accuracy in continuous letter recognition during air-writing gestures, and enhances accurate intent inference and flexible command mapping. The system processes air-writing gestures and environmental context, then uses LLMs to correct decoded gesture text and infer user intent, ultimately mapping it to executable commands.

Specifically, the system employs a camera-based recognition module for gesture capture, combined with a context recognition module that analyzes the environment using a VLM and object detection. The decoded gesture text and contextual information are input into the LLM, which infers the correct gesture input and user intent, and then maps the most relevant command and parameters. This enables flexible robot control, overcoming the limitations of predefined gestures and providing a more adaptive interaction framework. In summary, our main contributions are:

- 1) Proposed a novel gesture-based interaction framework, GesFlexiBot, that enables natural robot control through air-writing gestures;
- 2) Enhanced continuous letter decoding and user intent understanding in air-writing gestures by integrating environmental context Recognition and LLM inference;
- 3) Designed a promising method for resolving ambiguous relationships between personalized intents and executable commands, thereby improving the system's flexibility.

2 RELATED WORK

To enable robot control through gestures, three steps are involved: gesture input for intent expression, gesture intent inference, and intent-to-command mapping. In 2.1, we discuss existing gesture recognition methods and their potential for expressing complex intents. In 2.2, we review how user intent can be inferred from these

gestures. In 2.3, we discuss how the inferred intent is accurately mapped to specific system commands.

2.1 Gesture Input for Intent Expression

The gesture input ranges from abstract action gestures to symbolic gestures, each with its strengths and limitations in expressiveness, learning cost, and applicability [8],[16],[4].

Hosseini et al. [16] analyzed 1,728 gestures across 172 studies, finding that only 17.86% achieved consistent user understanding above 0.5. Moreover, most gestures are designed for simple tasks like "Confirm" or "Select", and require users to memorize specific gestures for each application, increasing cognitive load. For instance, Li et al. [20] introduced the Voice-Accompanying Hand-to-Face gesture for system commands. This approach works well for simple tasks in specific scenarios but presents challenges when users need to apply multiple specific gestures across diverse tasks, leading to a significant cognitive load.

Some studies address the memorability issue. Feng et al. [11] proposed a Many-to-One mapping approach, allowing similar gestures to represent the same command, increasing flexibility. [15],[27] have also explored diverse gesture expressions for the same command. However, these methods are limited to specific devices, such as screens or smart rings. At the same time, they still struggle with complex tasks and rely on predefined gestures, requiring users to learn specific actions for each command.

To support more flexible expression, some studies map abstract action gestures to the alphabet. However, users are required to learn a new encoding system, which, in turn, reduces usability. Typing in the air is another example of this method [10],[9].

In comparison, air-writing gestures provide greater expressive freedom and eliminate memory load, as they allow users to freely write letters in space without physical contact. Considering the general applicability of scenarios, we focus exclusively on device-free air-writing recognition methods, which can be further categorized into vision-based and radio-based approaches. The former utilizes 2D or 3D cameras to capture gesture input images, while the latter employs radio sensors such as radar [3], [19], [32] or WiFi [12], [14] to acquire gesture signals. Given that radio sensors are less prevalent than cameras and their positioning is not controllable, we limit our scope to camera-based methods.

The majority of camera-based methods are capable of offering accurate single-character recognition [17], [28], [31], [22]. However, recognizing continuous letters in full words or phrases remains a challenging task, as the positions of individual characters and the connections between them are both uncertain. Some studies utilize 3D cameras to capture depth information, thereby segmenting writing actions; however, 3D systems are more complex and incur higher costs [2]. Other research employs delimiters or constraints on stroke directions, but this approach impacts the fluency of user input [5],[24]. At the same time, although these studies often achieve recognition accuracies exceeding 90%, they face challenges in detecting naturally written continuous letters or words. The one that aligns most closely with our expectations is the work proposed by Kim et al. [18], which achieves a 29% character error

rate (CER) for English words. Nonetheless, the accuracy of recognition still requires improvement for real-world applications, with word correction being necessary.

2.2 Inference Methods for Gesture Intents

Predefined gesture interactions typically do not involve intent inference. Instead, specific gestures are mapped directly to system tasks, with the gesture itself representing the user's intent. For example, a "swipe left" gesture corresponds to the "go back" intent [16].

However, intent inference from gestures remains widely explored. Some studies explored natural gestures, which are natural hand movements without predefined structures. For example, [23] uses micro-movements like hand tension and orientation to infer emotional states. However, such methods are limited to emotional tendencies and do not provide explicit user intent.

Other research integrates contextual information for more accurate intent inference. For instance, [25] combines gaze, hand movements, and the arrangement of objects to infer the user's selected arrangement. PhotoBot [21] uses a VLM to analyze the environment and combine it with user input (pose and voice) to infer the desired photography pose. Similarly, [30] combines gesture recognition and object identification to determine whether the user wants the robot to move closer or further away.

2.3 Mapping Methods from Intents to System Commands

As discussed in 2.2, when a predefined mapping exists between user input and a set of system commands, intent inference and command mapping are unnecessary [8],[16],[4],[11],[15]. However, in scenarios where user can freely express their intents, these intents often do not directly correspond to specific system commands, requiring the system to identify the most relevant command combinations. For example, the user's intent might be "move the object from right to left," but no command explicitly named "move object from right to left" exists. In such cases, the system must search for and select the best-matching commands.

Some methods can be referred to apply for command matching. Traditional approaches, like rule-based and machine learning methods, are fast and controllable, making them suitable for large-scale data matching [26],[13],[1],[6]. In contrast, LLM/VLM methods excel in complex reasoning and contextual understanding, especially when intent and command do not align perfectly. For instance, [21] used a sentence transformer to match target and desired image descriptions by computing cosine similarities, while [29] employed VLM for similar image-matching tasks.

In summary, we have analyzed the techniques and methods for translating "gesture-based intent expression" into "system-executed commands." We find that combining air-writing gestures with large models enhances flexibility and accuracy, providing an optimal solution for intent expression and command mapping.

3 METHODS

Our proposed pipeline consists of two modules: a sensing component that captures and decodes air-writing gestures and contextual information, and an inference component that infers user intent

and maps it to executable system commands with required parameters. An overview of the system is shown in Fig.2. We detail the air-writing gesture recognition process in Section 3.1, the contextual understanding method in Section 3.2, the user intent inference in Section 3.3, and the command mapping in Section 3.4.

3.1 Air-writing Gesture Recognition

To decode air-writing gestures into meaningful text, we adopt the method proposed in [18], which utilizes spatio-temporal convolutional neural networks (ST-CNNs) to process video sequences of finger movements. Among the evaluated architectures, we select the Spatio-Temporal Residual 3D Network (ST-R3D) for its superior ability to simultaneously extract spatial and temporal features. This architecture demonstrates robust performance for English letter recognition, effectively handling unconstrained finger trajectories in free space, and ensuring accurate recognition across diverse input scenarios.

Air-writing gestures are recorded using an RGB camera positioned to capture finger trajectories against various backgrounds. Each gesture input is represented as a video sequence consisting of n frames, where each frame corresponds to a specific moment in the user's finger motion. Preprocessing steps include resizing the frames to a fixed resolution and standardizing the temporal alignment to ensure consistency across inputs. The preprocessed video is then fed into the Spatio-Temporal Residual 3D Network (ST-R3D), which extracts spatio-temporal features to represent the finger motion patterns.

After feature extraction, the output from the ST-R3D model is passed to the Connectionist Temporal Classification (CTC) loss function for decoding. It aligns the predicted character probabilities with the ground-truth text, enabling end-to-end decoding without requiring frame-level annotations. During inference, the system generates a sequence of character probabilities, which are decoded into coherent text using CTC alignment. For example, a gesture producing probabilities $[h, -, e, -, l, l, -, o]$ is decoded as "hello." The hand gesture trajectories captured by the camera and their corresponding decoded text are shown in Fig.3.

The character error rate (CER) is used to evaluate recognition performance, measures the accuracy of the decoded text, and is defined as:

$$CER = \frac{MCD(S, P)}{\text{length}_c(P)} \times 100(\%)$$

where $MCD(S, P)$ is the minimum character distance (Levenshtein measure) between the decoded phrase S and the ground-truth phrase P , and $\text{length}_c(P)$ is the number of characters in P . The Levenshtein measure counts the number of insertions, deletions, and substitutions required to transform S into P .

The WiTA achieved a CER of 29.24% in the WiTA English dataset, which is acceptable for a gesture recognition method, though the decoded text remains insufficiently accurate. To address this limitation, we propose leveraging LLMs. The decoded text will be treated as the user input and further revised in the intent inference stage (Section 3.3). This input correction process ensures a more precise understanding of the user's intent, thereby enhancing the accuracy of intent inference.

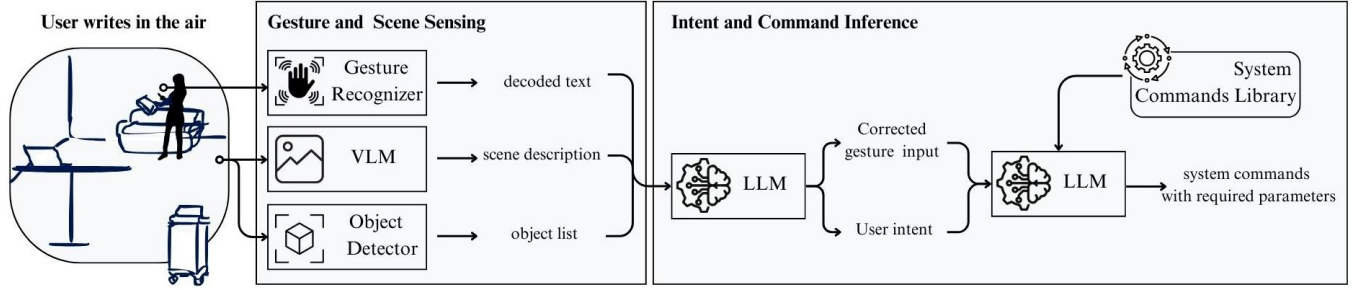


Figure 2: GesFlexiBot system diagram. The two main modules, the Sensing Module and the Inference Module, empower GesFlexiBot to recognize and interpret free-form air-writing gestures, enabling flexible and intuitive robot control.



Figure 3: (Top) Hand gesture trajectories during air-writing. (Bottom) Examples of writing patterns and their corresponding decoded text[18].

3.2 Contextual Information Recognition

We extract a list of objects in the environment that may interact with the user’s intent, clarifying and supplementing any potential ambiguities, which helps in intent inference. Additionally, we recognize the user’s environment and generate a scene description to help the LLM understand the contextual background of the user’s intent.

Specifically, to generate a more detailed contextual description of the current view, we use Detic [33] as our object detector to extract a list of objects. Detic expands the vocabulary of traditional object detectors by training on image classification datasets, enabling the recognition of tens of thousands of concepts beyond the capabilities of conventional detection models. This allows Detic to identify objects in diverse, unconstrained environments. For a specific frame from the user’s gesture video, Detic detects objects and outputs a list such as [“person”, “sofa”, “book”, “table”, “laptop”].

Then, we use InstructBLIP [7] as our VLM to create a visual scene description of current view. It provides semantic context by describing the relationships between objects and their spatial arrangement. For example, in the scene shown in Fig. 4, InstructBLIP might generate the following description: “The image shows a person standing next to a sofa, holding a book in her hand. To the person’s front left, there is a wooden table with an open laptop on it. The space seems to be a study area”.

Both the visual scene descriptions and object lists are then combined into a single textual context information, providing comprehensive environmental context for the subsequent intent inference and command mapping.

At execution time, we create a textual embedding of the user prompt, which consists of the user’s gesture input from Step 2 and the scene description with detected objects for the current view. This user prompt will be used for gesture input correction and intent inference in Section 3.3.

3.3 Gesture Input Correction and Intent Inference

After capturing the user’s gesture and contextual information from the current view, the system proceeds to the gesture input correction and intent inference steps.

The initial decoded gesture input may contain errors or ambiguities due to factors such as user variability, noise in the input signal, or incomplete gesture recognition. To improve the accuracy of the user’s intent, we first correct the gesture input, ensuring that the input aligns with the user’s actual intent.

We use GPT-4 for gesture input correction. It takes the decoded gesture text and the contextual information as input, and refines the text to better match the user’s intended meaning. For instance, if a gesture intended to express “put to table” is decoded as “puttotdble,” the LLM can detect spelling errors and correct them by comparing the input with world knowledge and contextual information. Through this process, the system resolves potential ambiguities in the gesture input, producing a more accurate representation of the user’s intent. Compared to traditional spelling correction, this approach avoids correcting the gesture input to words that are similar to the erroneous input but unrelated to the possible intent.

The corrected user input is then used for intent inference. At this stage, the system uses both the corrected gesture input and contextual information to infer most likely user intent. Intent inference is also performed by GPT-4, which generates the user’s intended tasks. The gesture input correction and intent inference are performed simultaneously by the LLM using an intent inference prompt, with the results being output separately, as shown in Figure 4.

3.4 System Command Mapping

After the user’s intent has been inferred, the next step is to map the inferred intent to the appropriate system command and generate the required parameters. The system takes the user’s inferred intent and the list of available commands for the robot as inputs, also using GPT-4 with a command mapping prompt to first match the most relevant system command that aligns with the user’s intent. Once

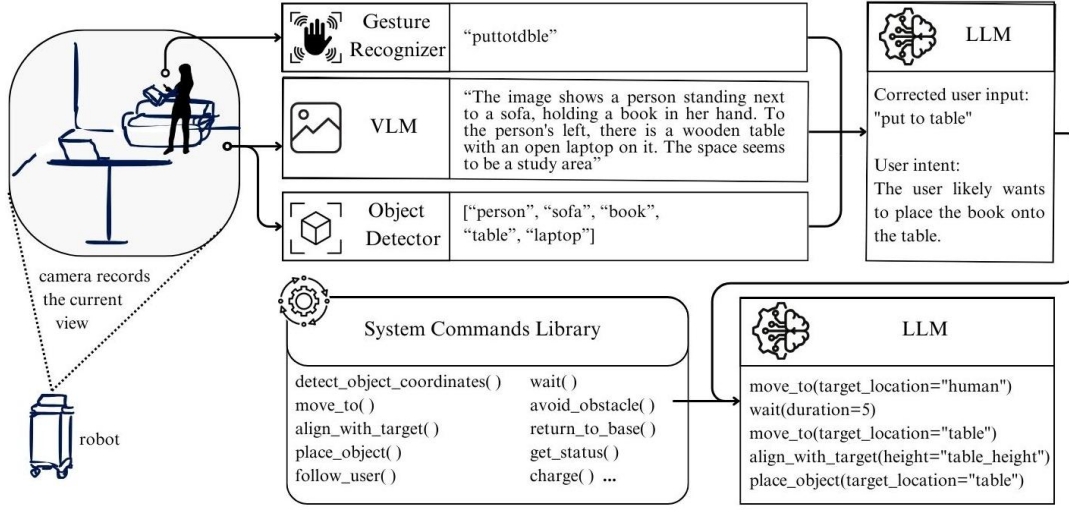


Figure 4: An example of using the GesFlexiBot system to translate air-writing gestures into executable system commands.

the matching command is identified, the system then refines the required parameters needed to execute the command, based on the specifics of the user's intent.

For example, when the intent "The user likely wants to place the book onto the table" is inferred, the system must map this intent to a sequence of suitable commands that guide the robot through the necessary actions. The relevant system commands for this intent could be: `move_to()`, `wait()`, `align_with_target()`, and `place_object()`. The parameters required for these commands include the user's coordinates, the duration of waiting while the human places the book, and the table's coordinates and height. Therefore, the complete set of commands could be: `move_to(target_location="user")`, `wait(duration=5)`, `move_to(target_location="table")`, `align_with_target(height="table_h")`, and `place_object(target_location="table")`.

The mapping of user intents to system commands is essential for translating the high-level, natural language-based gestures into low-level control commands that can be executed by the robot. By combining intent inference with command matching and parameter refinement, the system can accurately execute complex tasks with flexible user input.

In summary, the system completes the entire process of "air-writing gesture recognition, contextual information extraction, gesture input correction, intent inference, and command mapping," enabling the robot to perform actions based on diverse and changing user gestures.

4 EVALUATION AND FUTURE WORK

Different robots are equipped with varying functions and, consequently, different sets of system commands. Therefore, we consider three types of robots and define their assumed functionalities to evaluate GesFlexiBot's ability to operate across different robot platforms, as shown below:

- R1: Educational Robot, designed to answer questions and play music.
- R2: Service Robot, equipped with navigation, target following, information retrieval, and message-sending functions.

- R3: Assembly Robot, capable of navigation, picking up objects, and placing objects.

Each robot is equipped with a camera. In our experiments, the rear camera of an iPhone 12 Pro Max was used as a substitute for the robot's camera. Three tasks are designed to cover key aspects of human-robot interaction: Conversational Interaction, Action Command, and Object Handover. Conversational Interaction involves gesture-based communication, where the robot responds to user gesture input without requiring spatial awareness. Action Command interprets gesture input as movement directives for dynamic interaction. Object Handover requires the robot to perceive and deliver objects in the environment based on user intent. These tasks evaluate GesFlexiBot's ability to map free-form gestures to diverse system commands across different robotic functions.

For each task, a specific intent is predefined for testing, allowing users to express the same intent through different gesture-based textual inputs. These tasks and intents are detailed in Table 1.

Table 1: Tasks and intents for testing

Task	User Intent
T1: Conversational Interaction	Tell me a story
T2: Action Command	Follow me
T3: Object Handover	Take the book to me

We proceeded with Task T1 using Robot R1, Task T2 with R2, and Task T3 with R3. Each task scenario involved 2 participants ($n = 2$) performing air-writing gestures as a preliminary test. Based on the given intent, users were free to choose the word or phrase they wished to write.

Several representative test scenarios and the gesture input texts written by the user are shown in Fig. 5. First, we evaluated whether GesFlexiBot could accurately correct letter errors in the decoded gesture text and infer the corresponding user intent. Then, we assessed whether GesFlexiBot could accurately identify the appropriate command combinations and generate the required parameters. The test results are also presented in Fig. 5.

	T1 with R1	T2 with R2	T3 with R3
Current View			
User Real Input	"tell me a story"	"follow me"	"I want book"
Decoded Gesture Input	"eellmeastoiy"	"followne"	"wvantbook"
Object List	["person"]	["person", "door", "door"]	["person", "book", "key", "plant"]
Scene Description	"A young woman sits in front of a green background, wearing a gray jacket with cartoon graphics. The jacket's zipper is pulled up to the chin, and the person points forward with a focused expression. The background is a plain wall, creating a simple composition."	"A young woman wearing a face mask and a striped jacket stands in front of a white wall. She raises one hand in a pointing gesture. Behind her, there are two closed wooden doors labeled '102', suggesting a hallway or corridor setting."	"A young woman wearing a striped jacket and a face mask stands in front of a white wall. She raises one hand in a pointing gesture. A plant is hanging on the wall to her right. In the foreground, a book and a key are placed on a reflective surface."
Corrected Gesture Input	"tell me a story"	"follow me"	"want book"
Inferred Intent	"The user wants to hear a story."	"The user wants the robot to follow them."	"The user wants the robot to bring her the book from the nearby table."
Inferred Commands	generate_text(prompt) prompt = "Tell a story"	detect_target(target_type="human 1") follow_target()	detect_target(target_name="book") scan_area() navigate_to(x="table_x", y="table_y") pick_up(object_id="book_id", x="table_x", y="table_y") navigate_to(x="user_x", y="user_y") place(object_id="book_id", x="user_x", y="user_y")

Figure 5: The results generated by GesFlexiBot.

It can be seen that all tasks accurately inferred the user's intent and mapped it to the correct commands. Although the gesture input correction was not always perfect—for example, the correction for 'I want book' was 'want book'—the intent inference successfully understood the user's needs, so it did not affect the subsequent command mapping.

The preliminary tests demonstrate that GesFlexiBot has the potential to expand users' freedom of expression in gesture-based interaction, enabling more flexible and natural human-robot interaction. However, several potential failure cases remain. First, in highly cluttered backgrounds, the accuracy of gesture decoding decreases. Second, while contextual information recognition and LLM-based input correction significantly improve errors in gesture decoding, they do not guarantee perfect correction. This may lead to inference errors between similar words. For example, "cup" may be misrecognized as "cap" during the gesture decoding stage. Although the LLM can leverage world knowledge to correct "cap with tea" to "cup with tea", if the gesture is decoded as "give me cap" and both a cup and a cap are detected in the environment, the LLM may fail to resolve this ambiguity. Additionally, contextual information recognition may introduce irrelevant details. For instance, in the intent "tell me a story", objects detected in the environment, such as a cup or a cap, might be unnecessarily incorporated into the robot's generated story. While such additions could be surprising and engaging, they may also distort the user's intended request.

Thus, further testing and improved intent clarification mechanisms are necessary. In the next phase, we will deploy GesFlexiBot on physical robots for extended evaluation and attempt to clarify user intent through further interactions, such as additional queries when two or more intents with similar probabilities are inferred during LLM inference.

5 CONCLUSION

In this paper, we introduced GesFlexiBot, a flexible gesture-based robot interaction system that enables natural control through air-writing gestures. By combining VLMs and LLMs, GesFlexiBot can correct user gesture inputs, infer their intents, and map them to executable commands. The system's ability to understand free-form gestures and integrate environmental context presents a promising approach for more flexible human-robot interaction. Preliminary experiments demonstrated that GesFlexiBot can successfully process user inputs and generate corresponding commands, highlighting the feasibility of this solution. While further testing is needed, these initial results suggest significant potential for future development, offering a promising step toward more intuitive and adaptable gesture-based human-robot collaboration.

References

- [1] Emre Akdemir and Necaattin Barışçı. 2024. A review on deep learning applications with semantics. *Expert Systems with Applications* 251 (2024), 124029. doi:10.1016/j.eswa.2024.124029
- [2] Md. Shahinur Alam, Ki-Chul Kwon, Md. Ashraful Alam, Mohammed Y. Abbas, Shariar Md Imtiaz, and Nam Kim. 2020. Trajectory-Based Air-Writing Recognition Using Deep Neural Network and Depth Sensor. *Sensors* 20, 2 (2020). doi:10.3390/s20020376
- [3] Muhammad Arsalan and Avik Santra. 2019. Character Recognition in Air-Writing Based on Network of Radars for Human-Machine Interface. *IEEE Sensors Journal* 19, 19 (2019), 8855–8864. doi:10.1109/JSEN.2019.2922395
- [4] Alessandro Carfi and Fulvio Mastrogiovanni. 2023. Gesture-Based Human-Machine Interaction: Taxonomy, Problem Definition, and Analysis. *IEEE Transactions on Cybernetics* 53, 1 (2023), 497–513. doi:10.1109/TCYB.2021.3129119
- [5] Mingyu Chen, Ghassan AlRegib, and Bing-Hwang Juang. 2016. Air-Writing Recognition—Part I: Modeling and Recognition of Characters, Words, and Connecting Motions. *IEEE Transactions on Human-Machine Systems* 46, 3 (2016), 403–413. doi:10.1109/THMS.2015.2492598
- [6] Xiaojun Chen, Shengbin Jia, and Yang Xiang. 2020. A review: Knowledge reasoning over knowledge graph. *Expert Systems with Applications* 141 (2020), 112948. doi:10.1016/j.eswa.2019.112948

- [7] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. 2024. InstructBLIP: towards general-purpose vision-language models with instruction tuning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '23). Curran Associates Inc., Red Hook, NY, USA, Article 2142, 18 pages.
- [8] Kshitij Deshpande, Varad Mashalkar, Kaustubh Mhaisekar, Amaan Naikwadi, and Archana Ghotkar. 2023. Study and Survey on Gesture Recognition Systems. In *2023 7th International Conference On Computing, Communication, Control And Automation (ICCUBEA)*. IEEE, 1–6. doi:10.1109/ICCUBEA58933.2023.10392214
- [9] John J. Dudley, Jingyao Zheng, Aakar Gupta, Hrvoje Benko, Matt Longest, Robert Wang, and Per Ola Kristensson. 2023. Evaluating the Performance of Hand-Based Probabilistic Text Input Methods on a Mid-Air Virtual Qwerty Keyboard. *IEEE Transactions on Visualization and Computer Graphics* 29, 11 (2023), 4567–4577. doi:10.1109/TVCG.2023.3320238
- [10] Jacqui Fashimpaur, Kenrick Kin, and Matt Longest. 2020. PinchType: Text Entry for Virtual and Augmented Reality Using Comfortable Thumb to Fingertip Pinches. In *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI EA '20). Association for Computing Machinery, New York, NY, USA, 1–7. doi:10.1145/3334480.3382888
- [11] Shichang Feng, Zhiqian Feng, and Lijuan Cao. 2019. Many-to-One Gesture-to-Command Flexible Mapping Approach for Smart Teaching Interface Interaction. *IEEE Access* 7 (2019), 179517–179531. doi:10.1109/ACCESS.2019.2957365
- [12] Zhangjie Fu, Jiashuang Xu, Zhuangdi Zhu, Alex X. Liu, and Xingming Sun. 2019. Writing in the Air with WiFi Signals for Virtual Reality Devices. *IEEE Transactions on Mobile Computing* 18, 2 (2019), 473–484. doi:10.1109/TMC.2018.2831709
- [13] I. S. Gallage and D. I. De Silva. 2024. Exploring the Synergistic Relationship Between Syntax and Semantics in Rule-Based Translation. In *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. 1–6. doi:10.1109/ICCCNT61001.2024.10724832
- [14] Ruiyang Gao, Wenwei Li, Jinyi Liu, Shuyu Dai, Mi Zhang, Leye Wang, and Daqing Zhang. 2024. WiGesture: Meta-Motion-Based Continuous Gesture Recognition With Wi-Fi. *IEEE Internet of Things Journal* 11, 9 (2024), 15087–15099. doi:10.1109/IJOT.2023.3343875
- [15] Bogdan-Florin Gheran, Jean Vanderdonck, and Radu-Daniel Vatavu. 2018. Gestures for Smart Rings: Empirical Results, Insights, and Design Implications. In *Proceedings of the 2018 Designing Interactive Systems Conference* (Hong Kong, China) (DIS '18). Association for Computing Machinery, New York, NY, USA, 623–635. doi:10.1145/3196709.3196741
- [16] Masoumehsadat Hosseini, Tjado Ihmels, Ziqian Chen, Marion Koelle, Heiko Müller, and Susanne Boll. 2023. Towards a Consensus Gesture Set: A Survey of Mid-Air Gestures in HCI for Maximized Agreement Across Domains (CHI '23). Association for Computing Machinery, New York, NY, USA, Article 311, 24 pages. doi:10.1145/3544548.3581420
- [17] Chaur-Heh Hsieh, You-Shen Lo, Jen-Yang Chen, and Sheng-Kai Tang. 2021. Air-Writing Recognition Based on Deep Convolutional Neural Networks. *IEEE Access* 9 (2021), 142827–142836. doi:10.1109/ACCESS.2021.3121093
- [18] Ue-Hwan Kim, Yewon Hwang, Sun-Kyung Lee, and Jong-Hwan Kim. 2023. Writing in the Air: Unconstrained Text Recognition From Finger Movement Using Spatio-Temporal Convolution. *IEEE Transactions on Artificial Intelligence* 4, 6 (2023), 1386–1398. doi:10.1109/TAL.2022.3212981
- [19] Seong Kyu Leem, Faheem Khan, and Sung Ho Cho. 2020. Detecting Mid-Air Gestures for Digit Writing With Radio Sensors and a CNN. *IEEE Transactions on Instrumentation and Measurement* 69, 4 (2020), 1066–1081. doi:10.1109/TIM.2019.2909249
- [20] Zisu Li, Chen Liang, Yuntao Wang, Yue Qin, Chun Yu, Yukang Yan, Mingming Fan, and Yuanchun Shi. 2023. Enabling Voice-Accompanying Hand-to-Face Gesture Recognition with Cross-Device Sensing. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) (CHI '23). Association for Computing Machinery, New York, NY, USA, Article 313, 17 pages. doi:10.1145/3544548.3581008
- [21] Oliver Limoyo, Jimmy Li, Dmitriy Rivkin, Jonathan Kelly, and Gregory Dudek. 2024. PhotoBot: Reference-Guided Interactive Photography via Natural Language. doi:10.1109/IROS58592.2024.10801790 arXiv:2401.11061 [cs.CV]
- [22] Wei Lu, Zheng Tong, and Jinghui Chu. 2016. Dynamic Hand Gesture Recognition With Leap Motion Controller. *IEEE Signal Processing Letters* 23, 9 (2016), 1188–1192. doi:10.1109/LSP.2016.2590470
- [23] Yuhan Luo, Junnan Yu, Minhui Liang, Yichen Wan, Kening Zhu, and Shannon Sie Santos. 2024. Emotion Embodied: Unveiling the Expressive Potential of Single-Hand Gestures. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI '24). Association for Computing Machinery, New York, NY, USA, Article 404, 17 pages. doi:10.1145/3613904.3642255
- [24] Sohom Mukherjee, Sk. Arif Ahmed, Debi Prosad Dogra, Samarjit Kar, and Partha Pratim Roy. 2019. Fingertip detection and tracking for recognition of air-writing in videos. *Expert Systems with Applications* 136 (2019), 217–229. doi:10.1016/j.eswa.2019.06.034
- [25] Shouxiang Ni, Lindong Zhao, Ang Li, Dan Wu, and Liang Zhou. 2023. Cross-View Human Intention Recognition for Human-Robot Collaboration. *IEEE Wireless Communications* 30, 3 (2023), 189–195. doi:10.1109/MWC.018.2200514
- [26] Paramita Ray and Amlan Chakrabarti. 2022. A mixed approach of deep learning method and rule-based method to improve aspect level sentiment analysis. *Applied Computing and Informatics* 18, 1/2 (2022), 163–178. doi:10.1016/j.aci.2019.02.002
- [27] Yosra Rekik, Radu-Daniel Vatavu, and Laurent Grisoni. 2014. Match-up & conquer: a two-step technique for recognizing unconstrained bimanual and multi-finger touch input. In *Proceedings of the 2014 International Working Conference on Advanced Visual Interfaces* (Como, Italy) (AVI '14). Association for Computing Machinery, New York, NY, USA, 201–208. doi:10.1145/2598153.2598167
- [28] Zhou Ren, Junsong Yuan, Jingjing Meng, and Zhengyou Zhang. 2013. Robust Part-Based Hand Gesture Recognition Using Kinect Sensor. *IEEE Transactions on Multimedia* 15, 5 (2013), 1110–1120. doi:10.1109/TMM.2013.2246148
- [29] Dmitriy Rivkin, Gregory Dudek, Nikhil Kakodkar, David Meger, Oliver Limoyo, Michael Jenkin, Xue Liu, and Francois Hogan. 2023. ANSEL Photobot: A Robot Event Photographer with Semantic Intelligence. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. 8262–8268. doi:10.1109/ICRA48891.2023.10161403
- [30] Juan P. Vasconez, George A. Kantor, and Fernando A. Auat Cheein. 2019. Human-robot interaction in agriculture: A survey and current challenges. *Biosystems Engineering* 179 (2019), 35–48. doi:10.1016/j.biosystemseng.2018.12.005
- [31] Chong Wang, Zhong Liu, and Shing-Chow Chan. 2015. Superpixel-Based Hand Gesture Recognition With Kinect Depth Camera. *IEEE Transactions on Multimedia* 17, 1 (2015), 29–39. doi:10.1109/TMM.2014.2374357
- [32] Pengcheng Wang, Junyang Lin, Fuyue Wang, Jianping Xiu, Yue Lin, Na Yan, and Hongtao Xu. 2020. A Gesture Air-Writing Tracking Method that Uses 24 GHz SIMO Radar SoC. *IEEE Access* 8 (2020), 152728–152741. doi:10.1109/ACCESS.2020.3017869
- [33] Xingyi Zhou, Rohit Girdhar, Armand Joulin, Philipp Krähenbühl, and Ishan Misra. 2022. Detecting Twenty-Thousand Classes Using Image-Level Supervision. In *Computer Vision – ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part IX* (Tel Aviv, Israel). Springer-Verlag, Berlin, Heidelberg, 350–368. doi:10.1007/978-3-031-20077-9_21